

PROJECT TITLE: EDULODGE — CAMPUS HOSTEL & MESS MANAGEMENT SYSTEM

DOMAIN: SOFTWARE ENGINEERING / CORE JAVA APPLICATION

TECHNOLOGY STACK: JAVA (JDK 8+), OBJECT-ORIENTED DESIGN, JAVA SERIALIZATION, SHA-256 HASHING

NAME -ATHARV PATIL

REG NO.-25BAI10029

1. Executive Summary

Educational institutions require efficient systems to manage hostel occupancy, student records, and monthly mess operations. Manual record-keeping leads to data errors, duplicate room assignments, and insecure storage of user credentials.

EduLodge is an object-oriented desktop application built in Java to automate hostel administration. It features multi-role authentication, room allocation tracking, monthly mess bill computation, and file-based data persistence. The project demonstrates core software engineering principles including clean layered architecture, the Data Access Object (DAO) pattern, and cryptographic password security.

2. System Architecture & OOP Design

The application is structured around a four-tier architecture that enforces separation of concerns across data, logic, and interface layers:

Plaintext

[Presentation Layer] ---> MainApp (Console UI & Menu Controller)

|

[Service Layer] ---> AuthService | AllocationService | BillingService

|

[Data Access Layer] ---> GenericDAO (Interface) / FileGenericDAO (Binary Persistence)

|

[Domain Model Layer] ---> User (Abstract) —► Student / AdminWarden

Core Object-Oriented Principles Applied

- **Abstraction:** The User class and GenericDAO interface define abstract structures, hiding implementation details from caller services.
- **Inheritance:** Student and AdminWarden extend the abstract base class User, inheriting common user fields (userId, name, email, passwordHash) while defining specialized behavior.
- **Encapsulation:** All model attributes are private and accessed exclusively through public getter and setter methods.
- **Polymorphism:** The displayProfile() method is polymorphically overridden in both Student and AdminWarden to output custom user dashboards.

3. Class Hierarchy & Data Model

Class / Interface	Type	Role & Description
User	Abstract Class	Base class representing system users with authentication fields.
Student	Class (extends User)	Represents a student with roll number, assigned room, and mess active status.
AdminWarden	Class (extends User)	Represents a hostel warden with administrative access over block management.
Room	Class	Represents a hostel room with capacity limits, block location, and occupant IDs.
MessRecord	Class	Stores monthly attendance logs and daily rates for bill computation.
GenericDAO<T, ID>	Interface	Defines generic CRUD operations (save, findById, findAll, delete).

Class / Interface	Type	Role & Description
FileGenericDAO<T, ID>	Concrete Class	Implements GenericDAO using Java serialization (ObjectOutputStream/ObjectInputStream).
AuthService	Service Class	Handles student registration, credential verification, and SHA-256 password hashing.
AllocationService	Service Class	Manages room creation and enforces capacity limits during student assignment.
BillingService	Service Class	Computes monthly mess charges and exports text-formatted invoice receipts.
MainApp	Controller	Contains the main() entry point, interactive menus, and application workflow.

4. Key Functional Modules

4.1 Authentication & Security (AuthService)

User passwords are never stored in plain text. When a user registers or logs in, the input password is transformed using the **SHA-256 cryptographic hash function**. The resulting 64-character hexadecimal hash is stored and compared during authentication.

4.2 Room Allocation (AllocationService)

Hostel rooms have designated capacities (e.g., 2 occupants per room). When allocating a student:

1. The service checks whether the target room exists.
2. It verifies if the current occupant count is less than room capacity.
3. Upon success, the student ID is added to the room's list and saved to binary storage.

4.3 Mess Billing & Invoice Export (BillingService)

Mess bills are calculated dynamically using the formula:

$$\text{Total Bill} = \text{Days Attended} \times \text{Daily Rate}$$

The service can generate a local text file receipt (Invoice_<StudentID>_<Month>.txt) detailing line items for official student records.

4.4 File Persistence (FileGenericDAO)

System state is saved to binary .dat files (users.dat, students.dat, rooms.dat, mess.dat). On system start, objects are deserialized into memory, preserving all records across application restarts without an external SQL database server.

5. Software Testing & Verification

Test Case	Module	Input / Scenario	Expected Output	Status
TC-01	Registration	New Student registration with valid fields	Account created and saved to users.dat	PASS
TC-02	Login	Correct User ID & Password	Login successful, menu unlocked	PASS
TC-03	Security	Invalid Password on Login	"Authentication Error: Invalid password" message	PASS
TC-04	Allocation	Assign student to room with available capacity	Student room number updated; occupant added	PASS
TC-05	Overbooking	Assign student to a fully occupied room	"Allocation Error: Room is fully occupied"	PASS
TC-06	Billing	Attendance = 25 days, Daily Rate = ₹120.00	Total calculated bill = ₹3000.00	PASS
TC-07	Invoicing	Export invoice for logged record	Invoice text file generated locally	PASS

6. Future Scope

- **Database Migration:** Replace file-based DAO serialization with relational database storage (MySQL/PostgreSQL) via JDBC.
- **Graphical User Interface (GUI):** Transition from a console menu to an interactive graphical interface using JavaFX or Swing.
- **REST API Layer:** Expose business services as REST endpoints using Spring Boot to enable mobile application integration.

How to Save This Document as a PDF

- **Browser Method:** Press Ctrl + P (Windows) or Cmd + P (Mac), choose **Save as PDF** under the Destination dropdown, and save.
- **VS Code / Code Editor:** Open this response in VS Code, install the extension **Markdown PDF**, right-click anywhere in the file, and select **Markdown PDF: Export (pdf)**.